

Integrating Shopify and WooCommerce

Permissions, connection, endpoints, data, and database design

What this guide covers	
Permission	How a merchant authorizes your app on each platform, which scopes you need, and which approvals take time
Connection	Step-by-step OAuth (Shopify) and wc-auth (WooCommerce) flows, token storage, revocation
Endpoints	Exactly which API calls return orders, customers, items, fulfilments, refunds, attribution
Sync	Historical backfill, webhooks, idempotency, reconciliation
Database	Normalized PostgreSQL design, field mapping, upsert pattern, point-in-time queries
Compliance	Mandatory Shopify privacy webhooks, DPDP-style retention, tenant isolation

Prepared 3 October 2026. API facts were checked against Shopify and WooCommerce documentation on this date. Shopify's latest stable GraphQL Admin API version is 2026-07, and 2026-10 became stable on 1 October 2026. Always re-check version numbers and scope rules before you ship; both platforms change them regularly.

Contents

Contents	2
1. The big picture	4
1.1 Shopify and WooCommerce at a glance	4
1.2 The pipeline you are building	4
2. Shopify: accounts, apps and distribution	5
2.1 What to create	5
2.2 Choose a distribution method	5
2.3 Three ways to get an access token	5
3. Shopify: permissions (scopes) and approvals	5
3.1 Scopes for Merchant Co-Pilot (read-only)	6
3.2 The two approvals that take time	6
3.3 How the merchant grants (and withdraws) permission	6
4. Shopify: connecting a store (authorization code flow)	7
4.1 Step by step	7
4.2 HMAC verification of the install/callback query	7
4.3 Token types and lifetime	7
4.4 Declaring scopes and webhooks in shopify.app.toml (CLI apps)	8
5. Shopify: calling the API and what data you get	8
5.1 Making a request	8
5.2 Which object gives which data	9
5.3 Example: incremental order query	9
5.4 India-specific modelling problems you must solve	9
6. Shopify: historical backfill with Bulk Operations	10
7. Shopify: webhooks (live updates)	10
7.1 Headers you use	10
7.2 Handler rules	11
7.3 Mandatory privacy webhooks	11
8. WooCommerce: prerequisites and permission model	12
8.1 What the store must have before you can connect	12
8.2 Permission levels	12
8.3 How the merchant grants and withdraws access	12
9. WooCommerce: connecting a store	12
9.1 Option A: automatic key creation (recommended)	12
9.2 Option B: manual keys	13
9.3 Calling the API	13
10. WooCommerce: endpoints and the data they return	13
10.1 Endpoint map	13

10.2	Pagination and incremental sync	14
10.3	Statuses, COD and 'shipped'	14
10.4	Marketing attribution (UTM)	14
11.	WooCommerce: webhooks and keeping data fresh	14
11.1	Creating webhooks through the API	14
11.2	Topics and headers	15
11.3	Reliability problems you must design for	15
12.	Normalization: mapping both platforms to one schema	16
12.1	Orders	16
12.2	Customers, items and attribution	16
13.	Database design	17
13.1	Table groups and purpose	17
13.2	Idempotent upsert pattern (use for every ingest)	18
13.3	Tenant isolation	18
13.4	Indexing and growth	18
14.	Querying the stored data (examples)	18
14.1	RTO features at the Shipped moment (one order)	18
14.2	Churn features (RFM) for all customers at a scoring time	19
14.3	Business and health questions	19
15.	End-to-end flows and jobs	19
15.1	Onboarding a store	19
15.2	When an order is shipped (RTO prediction)	19
15.3	Learning from outcomes	20
15.4	Scheduled jobs	20
16.	Security, privacy and compliance	20
17.	A practical plan for your project	21
17.1	What to build first (in order)	21
17.2	Getting RTO labels for a college project	21
17.3	Pre-launch checklist	21
18.	Sources and things to re-verify	22
	Items I could not fully confirm	22

1. The big picture

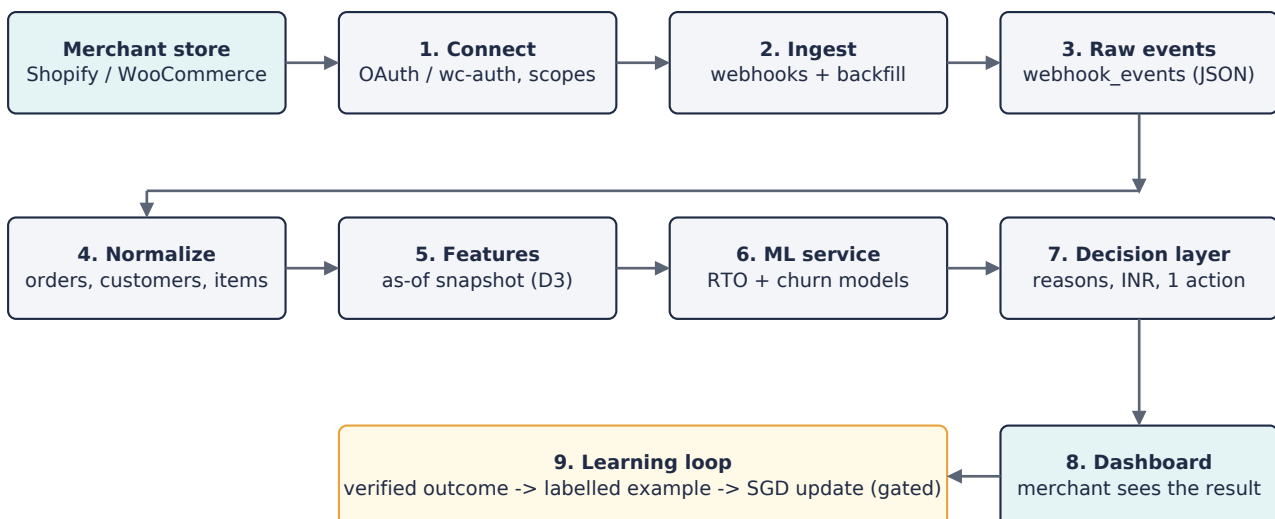
Merchant Co-Pilot reads a merchant's own store data and turns it into RTO and churn predictions. The most important rule, on both platforms, is this: **you can only read a store's data after the merchant (or the store admin) explicitly authorizes your app**. There is no public data feed of other stores, and scraping storefronts is against platform terms. The work is therefore: get authorization, pull data safely, store it in your own normalized database, and compute features from that database.

1.1 Shopify and WooCommerce at a glance

	Shopify	WooCommerce
What it is	Hosted platform. One central API, one app registry.	WordPress plugin. Every store is its own server.
How you get permission	Merchant installs your app and approves a scope list on Shopify's consent screen.	Merchant opens the wc-auth page on their own site and approves; the store generates an API key for you.
Credential you hold	Access token per shop (offline token; may expire and need a refresh token).	Consumer key + consumer secret per store (long-lived until the merchant deletes them).
API style	GraphQL Admin API (versioned, e.g. 2026-07). REST Admin API is legacy.	REST API v3 at /wp-json/wc/v3/.
Bulk history	Bulk Operations: async export to a JSONL file.	None. Paginated loops, 100 records per page.
Webhooks	Registered via config or API. HMAC header. Event ID for dedupe.	Created via API/admin. Signature header. Can be auto-disabled after failures.
Uninstall signal	app/uninstalled webhook, then shop/redact 48 h later.	None. You detect it when calls return 401/404.
Review / approval	Public App Store apps are reviewed. Extra approval for orders older than 60 days and for customer PII.	No central review. You depend on the merchant's host, plugins and WooCommerce version.

1.2 The pipeline you are building

Every connector, whatever the platform, feeds the same pipeline. Raw payloads are stored first, then normalized, then turned into point-in-time features. The model never sees raw platform JSON.



Practical tip. A student project does not need an App Store listing. Use a free Shopify **development store** and a free local or hosted WooCommerce site with generated test orders. Section 17 gives a practical plan.

2. Shopify: accounts, apps and distribution

2.1 What to create

- A free **Shopify Partner account** (partners.shopify.com). This lets you create apps and development stores.
- A **development store** from the Partner Dashboard. It is free, can hold test products/orders, and is where you test everything.
- An **app** in the **Dev Dashboard**, or with Shopify CLI (`npm init @shopify/app@latest`). The CLI template already implements OAuth, sessions and webhook handling, so use it if you can.
- Your app gets a **Client ID** and **Client secret**. The secret is also what verifies webhook signatures. Keep it server-side only.

2.2 Choose a distribution method

Method	Who can install	Review?	Use it when
Public (App Store)	Any merchant	Yes, Shopify reviews the app. Compliance webhooks are mandatory.	You are launching a real product.
Custom distribution	Specific stores you target with an install link	No App Store review	A handful of known merchants / pilot customers.
Your own organization's stores	Stores in the same Shopify organization as the app	No	Internal tools, your own dev stores.

2.3 Three ways to get an access token

Grant	Use when	Notes
Token exchange	Your app is embedded in the Shopify admin (App Bridge)	Recommended for embedded apps. Uses the session token; no redirect dance after install. Pair with Shopify-managed installation.
Authorization code grant	Your app is a standalone web app (own domain, own login)	Redirect merchant to Shopify, they approve, you exchange a code for a token. This is the likely fit for Merchant Co-Pilot's own dashboard.
Client credentials grant	Server-side app acting on stores in your own organization	Token is valid only 24 hours and must be re-requested. Fails with <code>shop_not_permitted</code> on any store outside your organization.

Changed recently. Apps created in the Dev Dashboard no longer show a copy-paste Admin API token in the UI. You get a Client ID and secret and obtain tokens through one of the grants above. Older tutorials that say 'copy the shpat_token from the admin' are out of date.

3. Shopify: permissions (scopes) and approvals

A **scope** is a permission string such as `read_orders`. You declare the scopes your app needs; the merchant sees them on the consent screen and approves or cancels. Request only what you use: fewer scopes means an easier review and more merchant trust. Any `write_*` scope also grants read access, so never request write for an analytics-only app.

3.1 Scopes for Merchant Co-Pilot (read-only)

Scope	Gives access to	Needed for	Approval
read_orders	Orders, line items, transactions, refunds, fulfilments on Order, last 60 days	order_value, COD flag, status, items	Merchant consent only
read_all_orders	Orders older than 60 days (used together with read_orders)	Training history for RTO and churn	Shopify must approve via Partner Dashboard 'Access requests'. Often takes weeks.
read_customers	Customer records, order counts, spend	RFM, churn features, repeat purchase	Protected customer data rules apply
read_products	Products, variants, categories	Product-level signals (future)	Merchant consent only
read_fulfillments (or the fulfilment-order scopes)	Fulfilment orders and shipment data	Shipped transition, carrier, tracking	Merchant consent; check which scope your chosen field needs
read_analytics / read_reports (optional)	ShopifyQL analytics	Aggregated sales/session numbers	Check current docs; not required for the core model

3.2 The two approvals that take time

- **60-day order window.** By default an app sees only the last 60 days of orders. Without `read_all_orders` you cannot build a proper training set. Request it early in the Partner Dashboard, explain that you need history to train per-store risk models, and plan for a wait.
- **Protected customer data.** Names, emails, phones and addresses are protected. You can add the scopes, but on non-development stores the API returns no protected data until your app is configured and approved for it. On *development* stores you can test without approval.

Design advice. Design the model so it works on the minimum data. Hash emails and phones on arrival, keep only state/city/PIN for geography, and never send names or phones to the ML service. This reduces both your compliance burden and your approval risk.

3.3 How the merchant grants (and withdraws) permission

- Merchant clicks your install link or finds your app in the App Store.
- Shopify shows the **consent screen** listing the access you requested.
- On approval Shopify redirects to your app with a one-time code (authorization code flow) or grants managed installation (embedded flow).
- **Withdrawal:** the merchant uninstalls the app. Shopify sends app/uninstalled immediately and the shop/redact compliance webhook 48 hours later. Your token stops working at uninstall.
- **Scope changes:** if you add scopes later, you must send the merchant back through authorization to approve them. After every token exchange, compare the granted scope with what you expect; a merchant can tamper with the scope in the URL.

4. Shopify: connecting a store (authorization code flow)

This is the flow for a standalone dashboard. If you use the CLI template, the library does all of it; read this to understand what the library is doing and what you must still handle (token storage, scope checks, uninstall).

4.1 Step by step

- **Install request.** Shopify (or your install link) sends the merchant to your app URL with shop, timestamp and hmac query parameters. Verify the hmac first (see 4.2).
- **Redirect to Shopify's grant screen** with your client_id, scopes, redirect_uri and a random state (nonce). Store the nonce in a signed cookie.
- **Merchant approves.** Shopify redirects to your redirect_uri with code, shop, state, hmac.
- **Validate the callback.** State equals your cookie; hmac valid; shop matches the myshopify.com hostname pattern (letters, digits, hyphen, dots only).
- **Exchange the code for a token** with a POST to the shop's access_token endpoint (see below).
- **Check granted scopes**, save the encrypted token, register webhooks, queue the historical backfill, redirect to your UI.

```
# 1) Redirect the merchant to the grant screen
https://{shop}.myshopify.com/admin/oauth/authorize
?client_id={CLIENT_ID}
&scope=read_orders,read_all_orders,read_customers,read_products
&redirect_uri=https://app.example.com/auth/callback
&state={RANDOM_NONCE}

# 2) After approval, exchange the code (server-to-server)
POST https://{shop}.myshopify.com/admin/oauth/access_token
Content-Type: application/json
{ "client_id": "...", "client_secret": "...", "code": "{code from redirect}" }

# Response (offline token) - store 'access_token' encrypted; compare 'scope' to what you asked for
{ "access_token": "shpat...", "scope": "read_orders,read_customers,..." }
```

4.2 HMAC verification of the install/callback query

```
// Node.js - verify Shopify's query-string hmac (NOT the same procedure as webhooks)
const crypto = require('crypto');
function verifyQueryHmac(query, clientSecret) {
  const { hmac, ...rest } = query;
  const message = Object.keys(rest).sort()
    .map(k => `${k}=${rest[k]}`).join('&');
  const digest = crypto.createHmac('sha256', clientSecret).update(message).digest('hex');
  return hmac && digest.length === hmac.length &&
    crypto.timingSafeEqual(Buffer.from(digest), Buffer.from(hmac));
}
const SHOP_RE = /^[a-zA-Z0-9][a-zA-Z0-9-]*\.myshopify\.com$/; // validate 'shop' before using it in a URL
```

4.3 Token types and lifetime

- **Offline token:** not tied to a logged-in user, usable by background jobs. This is what Merchant Co-Pilot needs for scheduled sync.
- **Online token:** tied to a staff user and expires after 24 hours. Not suitable for background syncing.
- **Expiring offline tokens and refresh tokens:** Shopify now can return a refresh token alongside the access token, with an expires_in value. Build your token table to hold *access_token*, *refresh_token*, *expires_at* and refresh before expiry. If Shopify returns a non-expiring token, the refresh columns simply stay empty.
- Store tokens **encrypted** (for example AES-256-GCM with a key from a secrets manager, plus a key-id column for rotation). Never log them and never send them to the browser.

4.4 Declaring scopes and webhooks in shopify.app.toml (CLI apps)

```
client_id = "YOUR_CLIENT_ID"
name = "merchant-copilot"
application_url = "https://app.example.com"
embedded = false

[access_scopes]
scopes = "read_orders,read_all_orders,read_customers,read_products"

[auth]
redirect_urls = ["https://app.example.com/auth/callback"]

[webhooks]
api_version = "2026-07"

[[webhooks.subscriptions]]
topics = ["orders/create", "orders/updated", "orders/cancelled", "fulfillments/create",
          "fulfillments/update", "refunds/create", "customers/create", "customers/update",
          "app/uninstalled"]
uri = "/webhooks/shopify"

[[webhooks.subscriptions]]
compliance_topics = ["customers/data_request", "customers/redact", "shop/redact"]
uri = "/webhooks/shopify/compliance"
```

5. Shopify: calling the API and what data you get

5.1 Making a request

```
POST https://{shop}.myshopify.com/admin/api/2026-07/graphql.json
X-Shopify-Access-Token: {access_token}
Content-Type: application/json

{ "query": "{ shop { name currencyCode ianaTimezone myshopifyDomain } }" }
```

- **Pin an API version** in the URL. Shopify releases a new version every quarter and supports each for about a year. As of 3 Oct 2026, 2026-07 is the latest stable and 2026-10 became stable on 1 Oct 2026. Pin one, test, and schedule an upgrade each quarter.
- **Rate limits are cost-based, not request-based.** Each query has a calculated cost; you have a bucket that refills at a fixed rate (sizes depend on the shop's plan). Every response includes `extensions.cost.throttleStatus`; use it to slow down, and request only the fields you need. For more than about 1,000 records use Bulk Operations (section 6).
- Always read `userErrors` and the top-level errors; GraphQL returns HTTP 200 even for many failures.

5.2 Which object gives which data

Data	GraphQL object / fields	Scope	Maps to your table
Shop settings	shop { name currencyCode ianaTimezone myshopifyDomain }	none	stores (currency, timezone)
Orders (header)	orders / Order: id, name, createdAt, updatedAt, cancelledAt, currencyCode, totalPriceSet, subtotalPriceSet, totalDiscountsSet, totalTaxSet, totalShippingPriceSet, displayFinancialStatus, displayFulfillmentStatus, paymentGatewayNames, tags	read_orders (+read_all_orders)	orders
Shipping geography	Order.shippingAddress { city province zip countryCodeV2 } (PII: needs protected-data access)	read_orders	orders.ship_city / state / postal_code
Line items	Order.lineItems { title sku quantity variant{id} product{id} originalUnitPriceSet discountedTotalSet }	read_orders	order_items
Payments	Order.transactions { gateway kind status amountSet }	read_orders	payment_status, payment_method_raw
Refunds	Order.refunds { id createdAt totalRefundedSet }	read_orders	orders.status (REFUNDED), outcome signals
Fulfillments	Order.fulfillments { status displayStatus createdAt inTransitAt deliveredAt trackingInfo { company number } }	read_orders / fulfillment scopes	fulfillments, orders.shipped_at, delivered_at
Customers	customers / Customer: id, createdAt, numberOfOrders, amountSpent, defaultAddress { city province zip }, email, phone	read_customers	customers
Attribution (UTM)	Order.customerJourneySummary { firstVisit lastVisit } with source and utmParameters	read_orders	customer_acquisition
Products	products / Product: id, title, productType, status, variants	read_products	order_items.category

Verify. Field names above come from the current GraphQL Admin schema, but fields move between versions (for example `LineItem` gained new price fields in 2026-07). Open the GraphQL explorer for *your pinned version* and copy field names from there before coding.

5.3 Example: incremental order query

```

query Orders($cursor: String, $q: String) {
  orders(first: 50, after: $cursor, query: $q, sortKey: UPDATED_AT) {
    pageInfo { hasNextPage endCursor }
    edges { node {
      id name createdAt updatedAt cancelledAt currencyCode
      displayFinancialStatus displayFulfillmentStatus paymentGatewayNames tags
      totalPriceSet { shopMoney { amount currencyCode } }
      shippingAddress { city province zip countryCodeV2 }
      customer { id }
      lineItems(first: 50) { edges { node { id title sku quantity } } }
      fulfillments { status createdAt deliveredAt trackingInfo { company number } }
    } }
  }
}
# variables: { "q": "updated_at: >= '2026-09-01T00:00:00Z'" }
```

5.4 India-specific modelling problems you must solve

- **There is no 'is_cod' field.** COD is just a payment gateway name chosen by the merchant (for example 'Cash on Delivery (COD)' or a custom label). Keep the raw string in `orders.payment_method_raw` and derive `is_cod` using a per-store list of patterns in `stores.config`. Ask the merchant to confirm the mapping during onboarding.
- **There is no 'RTO' status.** RTO normally comes from a courier or shipping app (tags, notes, or a separate courier API). Plan for three label sources: courier API, order tags the merchant applies, or a CSV import. Record the source in `outcomes.source`.

- **'Shipped' means a fulfilment was created.** Use the fulfilments/create webhook as the RTO prediction trigger and store its time as `orders.shipped_at`.
- **Guest checkout:** customer can be null. Fall back to a hash of email/phone to link repeat buyers.

6. Shopify: historical backfill with Bulk Operations

A new merchant may have tens of thousands of orders. Paging through them with normal queries would burn your rate-limit bucket. A **bulk operation** runs your query inside Shopify and gives you one downloadable JSONL file.

- Submit `bulkOperationRunQuery` with a query that uses connections (no top-level arguments for pagination).
- Wait for completion: subscribe to the `bulk_operations/finish` webhook (preferred) or poll the operation status.
- Download the url from the finished operation. It is JSON Lines: one object per line.
- Child records (line items, fulfilments) are separate lines carrying `__parentId`; re-attach them to the parent order while streaming.
- Concurrency limits per shop depend on the API version, so check the bulk operations guide for your pinned version. Run one export per shop at a time as a safe default.
- The download URL is signed and expires; fetch it promptly and stream it into your parser rather than loading it into memory.

```
mutation {
  bulkOperationRunQuery(query: ""
    { orders(query: "created_at:>=2025-01-01") { edges { node {
      id name createdAt updatedAt currencyCode paymentGatewayNames
      totalPriceSet { shopMoney { amount } }
      lineItems { edges { node { id title quantity sku } } }
      fulfillments { id status createdAt }
    } } } }
    """) { bulkOperation { id status } userErrors { field message } }
}
# JSONL output (conceptual):
# {"id":"gid://shopify/Order/1","name":"#1001",...}
# {"id":"gid://shopify/LineItem/9","quantity":2,"__parentId":"gid://shopify/Order/1"}
```

7. Shopify: webhooks (live updates)

After the backfill, webhooks keep your database current within seconds. Each delivery is an HTTPS POST with a JSON body.

7.1 Headers you use

Header	Use
X-Shopify-Hmac-Sha256	Base64 HMAC-SHA256 of the raw request body using your client secret. Verify before anything else; reject with 401 if wrong.
X-Shopify-Shop-Domain	Identifies the store, so you can look up <code>store_id</code> .
X-Shopify-Topic	Event type, e.g. <code>orders/create</code> .
X-Shopify-Event-Id	Stable ID for the event. Use it as <code>webhook_events.event_id</code> for deduplication.
X-Shopify-Webhook-Id	ID of this delivery attempt (retries differ).
X-Shopify-Triggered-At	When the event happened. Use it to ignore stale, out-of-order updates.

7.2 Handler rules

```
// Express: you MUST hash the raw bytes, so mount express.raw() on this route only
app.post('/webhooks/shopify', express.raw({ type: 'application/json' })), async (req, res) => {
  const sig = req.get('X-Shopify-Hmac-Sha256') || '';
  const calc = crypto.createHmac('sha256', CLIENT_SECRET).update(req.body).digest('base64');
  const ok = sig.length === calc.length && crypto.timingSafeEqual(Buffer.from(sig), Buffer.from(calc));
  if (!ok) return res.sendStatus(401);

  const shop = req.get('X-Shopify-Shop-Domain');
  const event = req.get('X-Shopify-Event-Id');
  const topic = req.get('X-Shopify-Topic');
  await db.query(
    // persist first; UNIQUE(store_id, event_id) makes retries harmless
    `INSERT INTO webhook_events(store_id,event_id,platform,topic,payload)
      SELECT store_id,$2,'shopify',$3,$4 FROM stores WHERE platform_shop_id=$1
      ON CONFLICT (store_id, event_id) DO NOTHING`, [shop, event, topic, JSON.parse(req.body)]);
  queue.add('process-webhook', { shop, event }); // do the heavy work asynchronously
  res.sendStatus(200);                          // respond fast (seconds), always after persisting
};
```

- Respond with 200 quickly and process in a queue. Slow responses cause Shopify to retry and eventually remove the subscription.
- Webhooks are **at-least-once and unordered**. Compare `updated_at` before overwriting, and run a periodic reconcile job that re-queries recently updated orders.
- Order and customer topics contain PII and need protected-customer-data approval on real stores.

7.3 Mandatory privacy webhooks

Topic	Meaning	What your system must do
customers/data_request	A customer asked the merchant for their data	Export what you hold for that customer (hashed IDs, orders, predictions) to the merchant within the legal window (Shopify expects 30 days).
customers/redact	Delete a customer's data	Null out or delete PII and customer-linked rows; keep only anonymized aggregates.
shop/redact	Sent 48 hours after the app is uninstalled	Delete everything for that store_id: connections, orders, features, predictions, model rows.

An invalid HMAC on a compliance webhook must return 401. Apps listed in the App Store fail review if these three are missing.

8. WooCommerce: prerequisites and permission model

WooCommerce has no app store, no central registry and no consent infrastructure of its own. The 'permission' is an **API key pair** that the store generates for a WordPress user. Your app holds that key; whoever owns the store can delete it at any time.

8.1 What the store must have before you can connect

- **HTTPS.** Keys are sent with HTTP Basic auth, so insist on a valid TLS certificate. Reject plain-HTTP stores.
- **Pretty permalinks.** The REST API lives under /wp-json/. If permalinks are set to 'Plain' you must use `?rest_route=/wc/v3/orders` instead.
- **A reasonably recent WooCommerce.** Use REST API **v3**. Read /wp-json/wc/v3/system_status at connect time to record the WooCommerce version, currency and whether high-performance order storage (HPOS) is on. HPOS is transparent to the REST API, so never query the database directly.
- **An administrator** who can open WooCommerce settings (only admins can create keys).
- **Not blocked by security plugins or a firewall.** Many hosts block /wp-json/ or the Authorization header. Your connect step should run a test call and show a clear error message when blocked.

8.2 Permission levels

Scope value	Allows	Your choice
read	GET only	Use this. Merchant Co-Pilot only recommends actions; it never edits orders.
write	POST / PUT / DELETE	Do not request.
read_write	Everything	Do not request. It also makes merchants less willing to approve.

8.3 How the merchant grants and withdraws access

- **Grant:** merchant approves on their own WooCommerce admin screen (section 9) or pastes keys you asked them to create.
- **Withdraw:** merchant deletes the key under WooCommerce > Settings > Advanced > REST API. Your calls begin to fail with 401/403. There is **no uninstall webhook**, so treat repeated 401/404 as 'disconnected', mark the connection revoked and stop syncing.
- **Privacy:** because there is no platform-level redact webhook, you must offer your own 'disconnect and delete my data' action and apply your retention policy.

9. WooCommerce: connecting a store

9.1 Option A: automatic key creation (recommended)

Send the merchant's browser to `{store_url}/wc-auth/v1/authorize` with these query parameters.

WooCommerce asks the logged-in admin to approve, then **POSTs the new keys to your callback_url** and sends the browser to your `return_url`.

Parameter	Meaning
app_name	Name shown to the merchant on the approval screen, e.g. 'Merchant Co-Pilot'.
scope	read, write or read_write. Use read .
user_id	Your own internal ID for this connection attempt (NOT a WooCommerce user ID). Use a one-time connection-attempt ID so you can match the callback.
return_url	Where the browser goes after the merchant approves or declines.
callback_url	Server endpoint that receives the generated keys. Must be HTTPS.

```
GET https://shop.example.com/wc-auth/v1/authorize
?app_name=Merchant+Co-Pilot&scope=read&user_id=ATTEMPT_7f3a
&return_url=https://app.example.com/connect/done
&callback_url=https://app.example.com/api/woo/callback

# WooCommerce then POSTs JSON to your callback_url:
{ "key_id": 12, "user_id": "ATTEMPT_7f3a", "consumer_key": "ck_xxxxxxx",
  "consumer_secret": "cs_xxxxxxx", "key_permissions": "read" }
```

- Match `user_id` to your stored attempt record; reject unknown or already-used attempts. Also check that the request really comes from the store domain you started with.
- Encrypt and save `consumer_key` and `consumer_secret` immediately. Do not log the callback body.
- Run a test call (GET /wp-json/wc/v3/system_status) and only then mark the store connected.
- The `consumer_secret` is shown only once. If you lose it, the merchant must create new keys.

9.2 Option B: manual keys

Merchant opens WooCommerce > Settings > Advanced > REST API > Add key, chooses description 'Merchant Co-Pilot', picks a user, permission **Read**, clicks Generate, then pastes the key and secret into your onboarding form. Use this as a fallback when the wc-auth screen is blocked by a plugin or theme.

9.3 Calling the API

```
curl -u ck_xxxxxxx:cs_xxxxxxx \
  "https://shop.example.com/wp-json/wc/v3/orders?per_page=100&page=1&orderby=date&order=asc"

# Node: npm i @woocommerce/woocommerce-rest-api
const api = new WooCommerceRestApi({ url, consumerKey, consumerSecret, version: 'wc/v3' });
const res = await api.get('orders', { per_page: 100, page: 1, modified_after: since, dates_are_gmt: true });
// res.headers['x-wp-totalpages'] tells you when to stop
```

10. WooCommerce: endpoints and the data they return

10.1 Endpoint map

Data	Endpoint (under /wp-json/wc/v3/)	Important fields	Maps to your table
Store settings	settings/general, system_status	woocommerce_currency, timezone, WC version, HPOS	stores
Orders	orders	id, number, status, currency, date_created_gmt, date_modified_gmt, date_paid, date_completed, total, discount_total, shipping_total, total_tax, customer_id, payment_method, payment_method_title, billing{}, shipping{}, line_items[], refunds[], meta_data[]	orders, order_items
Order notes	orders/{id}/notes	note text, date, customer_note flag	optional: shipment/RTO hints from plugins
Refunds	orders/{id}/refunds	id, date_created, amount, line_items	orders.status, outcome signals
Customers	customers	id, date_created, date_modified, email, first_name, billing{}, shipping{}, is_paying_customer, orders_count, total_spent	customers
Products	products, products/categories	id, name, sku, type, status, price, categories[]	order_items.category
Reports (limited)	reports/sales, reports/top_sellers	totals only	do not rely on; compute from orders
Webhooks	webhooks	id, topic, delivery_url, status, secret	integrations

10.2 Pagination and incremental sync

- `per_page` is capped at **100**. Read `X-WP-Total` and `X-WP-TotalPages` response headers.
- **Backfill:** loop pages with `orderby=date&=asc` and persist the last page/date as a cursor in `sync_jobs` so a crash resumes instead of restarting.
- **Incremental:** `modified_after=<ISO time>&dates;_are_gmt=true` returns only changed orders. Store the highest `date_modified_gmt` seen, minus a small overlap (a few minutes), and upsert so overlap is harmless.
- There is **no bulk export**. Small hosts time out on big pages: start with `per_page=50`, retry with exponential backoff on 429/5xx/timeouts, and throttle to one request at a time per store.
- Do not request more than you need. Use `_fields=id,status,total,...` to trim responses.

10.3 Statuses, COD and 'shipped'

- Built-in statuses: pending, processing, on-hold, completed, cancelled, refunded, failed (plus trash). Plugins add custom ones, so always keep the raw value in `platform_status`.
- **WooCommerce has no 'shipped' status.** Many stores use 'completed' for shipped; others use a custom status or a shipment-tracking plugin. During onboarding let the merchant choose which status (or which tracking-added event) means 'shipped', save it in `stores.config.shipped_status`, and set `orders.shipped_at` from the time the order first entered that status (read it from webhook arrival time or order notes).
- **COD detection:** the standard Cash on Delivery gateway has `payment_method = cod`. Keep `payment_method_title` too, because custom COD plugins use other IDs.
- **Guest orders** have `customer_id = 0`. Identify repeat guests by a hash of billing email/phone.

10.4 Marketing attribution (UTM)

WooCommerce's Order Attribution feature saves attribution as order meta with the `_wc_order_attribution_prefix` (for example `_wc_order_attribution_utm_source`, `_utm_medium`, `_utm_campaign`, `_source_type`). Read them from `meta_data[]`. It records only the visit that ended in checkout, can be empty for admin-created orders, and some stores report 'Unknown' because of caching/theme conflicts. Treat it as best-effort; ad spend itself comes from Google/Meta APIs or CSV upload, not from WooCommerce.

11. WooCommerce: webhooks and keeping data fresh

11.1 Creating webhooks through the API

```
POST https://shop.example.com/wp-json/wc/v3/webhooks (needs a key with write permission)
{ "name": "Co-Pilot order updated", "topic": "order.updated",
  "delivery_url": "https://app.example.com/webhooks/woo/STORE_ID",
  "secret": "<random 32+ chars, stored encrypted>", "status": "active" }
```

This is the one place a **read** key is not enough. Either ask for a `read_write` key only for the moment you register webhooks (then tell the merchant to downgrade it), or instruct the merchant to create the webhooks in WooCommerce > Settings > Advanced > Webhooks, or ship a tiny companion plugin that registers them. Decide this early because it changes your permission story.

11.2 Topics and headers

Item	Detail
Useful topics	order.created, order.updated, order.deleted, order.restored, customer.created, customer.updated, product.updated
X-WC-Webhook-Signature	Base64 HMAC-SHA256 of the raw body using the webhook secret. Verify it exactly as for Shopify, with the secret you set.
X-WC-Webhook-Source / Topic / Resource / Event	Which store sent it and what happened. Use Source to find the store_id.
X-WC-Webhook-ID / X-WC-Webhook-Delivery-ID	Webhook definition ID and delivery attempt ID.
Dedupe key	Delivery ID differs per attempt, so also dedupe on (store_id, order id, date_modified_gmt).

11.3 Reliability problems you must design for

- Deliveries run through WordPress's background scheduler (Action Scheduler) and can be delayed.
- After repeated delivery failures WooCommerce **disables the webhook** (the default threshold is five consecutive failures). Run a daily job that lists /webhooks, re-activates disabled ones, and recreates missing ones.
- Because webhooks can still be missed, run a **reconcile job** every few hours: call orders with modified_after and upsert. This also covers stores where webhooks cannot be created.
- If the merchant's hosting is slow or blocks callbacks, fall back to polling modified_after every 5 to 15 minutes.

12. Normalization: mapping both platforms to one schema

Write one adapter per platform behind an interface such as `EcommerceAdapter` (*connect*, *backfill*, *handleWebhook*, *fetchChangedSince*). Each adapter outputs the same normalized records. Nothing downstream knows which platform produced a row.

12.1 Orders

Normalized column	Shopify source	WooCommerce source
platform_order_id	Order.id (gid; store the number part)	id
placed_at	createdAt	date_created_gmt
platform_updated_at	updatedAt	date_modified_gmt
currency	currencyCode	currency
order_value	totalPriceSet.shopMoney.amount	total
subtotal / discount / tax / shipping	subtotalPriceSet / totalDiscountsSet / totalTaxSet / totalShippingPriceSet	(total - discount_total - shipping_total - total_tax) / discount_total / total_tax / shipping_total
payment_method_raw	paymentGatewayNames[0]	payment_method (+ payment_method_title)
is_cod	gateway name matches stores.config COD patterns	payment_method = 'cod' or matches COD patterns
platform_status	displayFinancialStatus + displayFulfillmentStatus	status
status (normalized)	mapped enum: PENDING, PROCESSING, SHIPPED, DELIVERED, CANCELLED, REFUNDED, RETURNED, RTO, UNKNOWN	same enum, using the store's shipped-status mapping
shipped_at	earliest fulfillment createdAt	time the order entered the merchant's 'shipped' status
delivered_at	fulfillment deliveredAt (if the carrier reports it)	date_completed only if the store uses it that way, else NULL
ship_city / state / postal_code	shippingAddress.city / province / zip	shipping.city / state / postcode
customer link	Order.customer.id, else hash(email/phone)	customer_id, else (0 = guest) hash(billing email/phone)

12.2 Customers, items and attribution

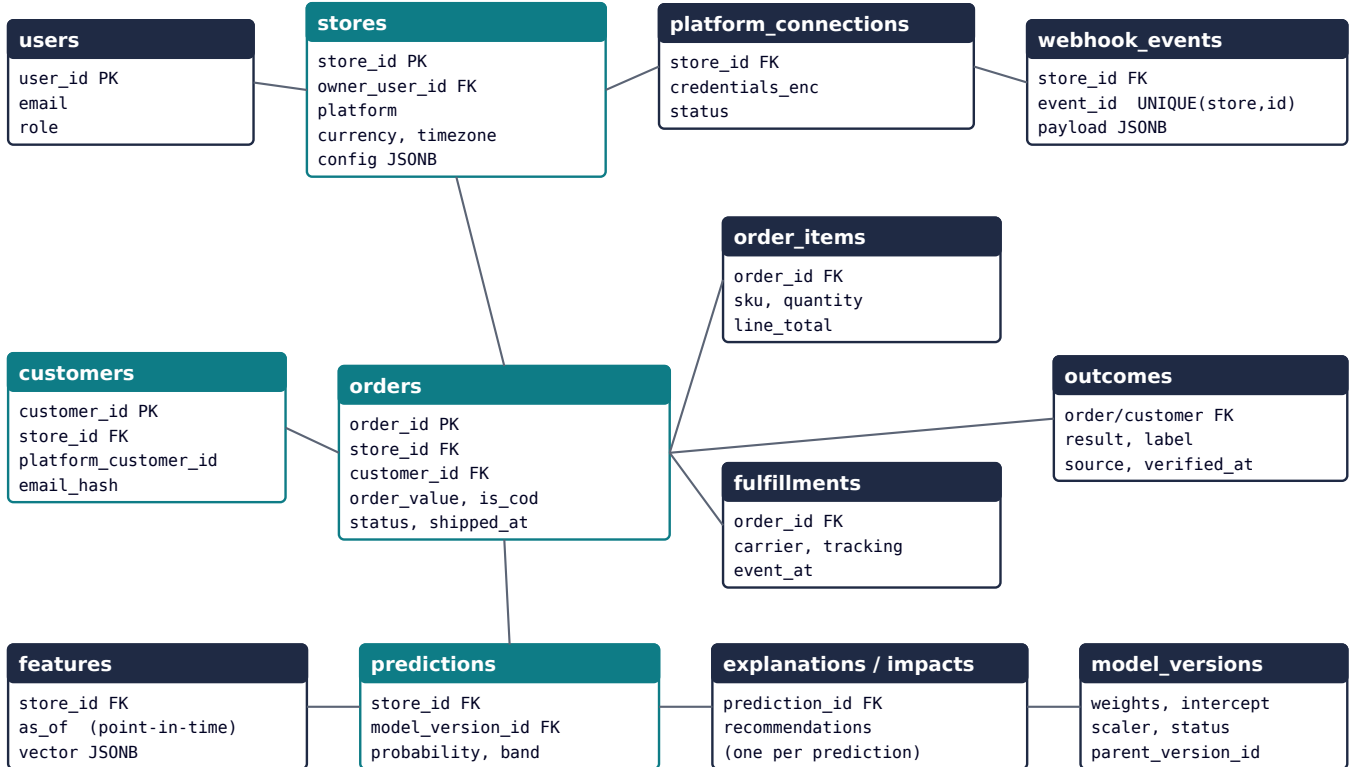
Table.column	Shopify source	WooCommerce source
customers.platform_customer_id	Customer.id	customers[].id (0 for guests: use hash key instead)
customers.email_hash / phone_hash	SHA-256 of lowercased email / E.164 phone	billing.email / billing.phone, hashed the same way
customers.platform_created_at	createdAt	date_created_gmt
order_items.sku / quantity / line_total	lineItems: sku, quantity, discountedTotalSet	line_items: sku, quantity, total
order_items.category	product.productType or collection	products/{id}.categories[0]
fulfillments.carrier / tracking_ref	trackingInfo { company number }	from shipment-tracking plugin meta (if installed)
customer_acquisition.utm_*	customerJourneySummary utmParameters	meta_data_wc_order_attribution_utm_*

Why. Keep the original payload. Store the raw JSON in `webhook_events.payload` (and `raw_meta` on orders). When you discover a mapping bug you can re-run normalization instead of re-fetching from the platform.

13. Database design

The design follows your project report (section 11) and the SQL file delivered earlier (merchant_copilot_schema_v2.sql, PostgreSQL 14+). Three principles drive it: **store_id on every tenant row**, **idempotent writes**, and **point-in-time features**.

Every tenant table carries store_id (teal = core path). Lines show foreign-key relationships.



13.1 Table groups and purpose

Group	Tables	Purpose / key constraints
Identity & tenancy	users, refresh_tokens, stores, store_members, categories	A user owns many stores. stores.platform + platform_shop_id is UNIQUE. stores.config (JSONB) holds thresholds, cost assumptions, COD patterns, shipped-status mapping.
Integration	platform_connections, webhook_events, sync_jobs	Encrypted credentials with key id; webhook_events UNIQUE(store_id, event_id) gives idempotency; sync_jobs stores cursors so backfills resume.
Normalized commerce	customers, orders, order_items, fulfillments	UNIQUE(store_id, platform_*_id) enables upsert. Composite FKs (store_id, order_id) make cross-store references impossible.
Labels	outcomes	One RTO label per order and one churn label per customer per scoring run; records source and verified_at.
Features & ML	feature_schemas, scoring_runs, features, training_runs, model_versions, model_updates, collective_rounds, collective_contributions	Feature contracts are frozen per version. Every prediction ties to a feature snapshot and model version. One SGD update per outcome.
Decisions	predictions, explanations, reason_templates, impact_formula_versions, impacts, rule_versions, rules, recommendations	Exactly one recommendation per prediction (UNIQUE). Only High/Medium predictions get explanations/impacts/recommendations.
Marketing	marketing_spend, customer_acquisition	Spend by channel/period and one attributed channel per customer.
Audit	audit_logs	Append-only; trigger blocks UPDATE/DELETE.

13.2 Idempotent upsert pattern (use for every ingest)

```
INSERT INTO orders (store_id, platform_order_id, customer_id, placed_at, platform_updated_at,
                    order_value, currency, payment_method_raw, is_cod, platform_status, status, raw_meta)
VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12)
ON CONFLICT (store_id, platform_order_id) DO UPDATE SET
    customer_id = EXCLUDED.customer_id, order_value = EXCLUDED.order_value,
    is_cod = EXCLUDED.is_cod, platform_status = EXCLUDED.platform_status,
    status = EXCLUDED.status, platform_updated_at = EXCLUDED.platform_updated_at,
    raw_meta = EXCLUDED.raw_meta
WHERE EXCLUDED.platform_updated_at >= orders.platform_updated_at; -- ignore stale, out-of-order events
```

13.3 Tenant isolation

- Resolve store_id from the authenticated session, never from the request body. At the start of each DB transaction run SET LOCAL app.store_id = '<uuid>'.
- Row-level security policies compare each row's store_id with that setting. FORCE ROW LEVEL SECURITY is on, and the application role must own none of the tables and must not have BYPASSRLS.
- Use a separate privileged role for cross-store jobs (the weekly FedAvg aggregation). Keep it out of the web process.
- Add CI tests that try to read, update and delete another store's rows and expect failure (your report's T-04).

13.4 Indexing and growth

- Indexes already in the schema: orders(store_id, placed_at), orders(store_id, customer_id, placed_at), orders(store_id, status, shipped_at), predictions(store_id, model_type, risk_band, predicted_at DESC).
- When orders pass tens of millions, partition orders/features/predictions by hash(store_id) or by month; keep webhook_events for 30 to 90 days, then archive.
- Store money as NUMERIC(14,2), timestamps as TIMESTAMPTZ in UTC, and convert to the shop timezone only when grouping by 'day'.

14. Querying the stored data (examples)

These queries show how your feature service reads the normalized tables. The golden rule: **every subquery is bounded by the prediction's as-of time**, so no future information leaks into a feature.

14.1 RTO features at the Shipped moment (one order)

```
-- $1 = store_id, $2 = order_id
SELECT o.order_id,
       o.order_value,
       o.is_cod::int AS cod_flag,
       (SELECT count(*) FROM orders p
        WHERE p.store_id = o.store_id AND p.customer_id = o.customer_id
        AND p.placed_at < o.placed_at) AS customer_order_count,
       COALESCE((SELECT avg(x.label::float) FROM outcomes x
                  JOIN orders p ON p.store_id = x.store_id AND p.order_id = x.order_id
                  WHERE p.store_id = o.store_id AND p.customer_id = o.customer_id
                  AND x.kind = 'rto' AND x.verified_at < o.shipped_at), 0) AS previous_rto_rate,
       COALESCE(EXTRACT(day FROM o.shipped_at - (SELECT max(p.placed_at) FROM orders p
                                                  WHERE p.store_id = o.store_id AND p.customer_id = o.customer_id
                                                  AND p.placed_at < o.placed_at)), 9999) AS purchase_recency_days
FROM orders o
WHERE o.store_id = $1 AND o.order_id = $2 AND o.is_cod AND o.shipped_at IS NOT NULL;
```

14.2 Churn features (RFM) for all customers at a scoring time

```
-- $1 = store_id, $2 = as_of (timestamp), $3 = observation window in days
SELECT c.customer_id,
       EXTRACT(day FROM $2 - max(o.placed_at)) AS recency_days,
       count(*) AS frequency,
       sum(o.order_value) AS monetary_value,
       EXTRACT(day FROM $2 - min(o.placed_at)) AS customer_tenure_days
FROM customers c
JOIN orders o ON o.store_id = c.store_id AND o.customer_id = c.customer_id
WHERE c.store_id = $1
      AND o.placed_at <= $2 AND o.placed_at > $2 - make_interval(days => $3)
      AND o.status NOT IN ('CANCELLED')
GROUP BY c.customer_id;
```

14.3 Business and health questions

```
-- COD vs prepaid RTO rate by state
SELECT o.ship_state, o.is_cod, count(*) AS orders,
       round(100.0 * avg(x.label), 1) AS rto_pct
FROM orders o JOIN outcomes x ON x.store_id = o.store_id AND x.order_id = o.order_id AND x.kind = 'rto'
WHERE o.store_id = $1 GROUP BY 1,2 ORDER BY orders DESC;

-- Webhooks that failed or are stuck
SELECT topic, status, count(*), max(received_at) FROM webhook_events
WHERE store_id = $1 AND status IN ('failed','dead','received')
GROUP BY 1,2 ORDER BY 3 DESC;

-- Stores whose connection needs attention
SELECT s.name, c.status, c.last_synced_at, c.last_error
FROM platform_connections c JOIN stores s USING (store_id)
WHERE c.status <> 'active' OR c.last_synced_at < now() - interval '1 day';
```

15. End-to-end flows and jobs

15.1 Onboarding a store

Step	Shopify	WooCommerce
1. Merchant enters store	myshopify domain (validated by regex)	Store URL (must be HTTPS)
2. Authorize	Redirect to grant screen; merchant approves scopes	Redirect to /wc-auth/v1/authorize; merchant approves read key
3. Receive credential	Callback with code, exchange for token	POST callback with consumer key/secret
4. Verify	Check granted scopes, call shop { ... }	Call system_status; record version, currency, HPOS
5. Save	Encrypt token, create stores + platform_connections rows	Encrypt keys, create rows
6. Register webhooks	From TOML config or webhookSubscriptionCreate	Create via API / merchant / companion plugin
7. Configure mappings	COD gateway patterns	COD patterns + which status means 'shipped'
8. Backfill	Bulk operation, then JSONL import	Paginated order/customer loops (resumable)
9. Go live	Webhooks + hourly reconcile	Webhooks + 15 min to 3 h reconcile

15.2 When an order is shipped (RTO prediction)

- Webhook arrives, signature verified, event row inserted (duplicates ignored), HTTP 200 returned.
- Worker normalizes the order and upserts orders/order_items/fulfillments, setting shipped_at.
- If is_cod and shipped_at just became non-null, enqueue 'score-rto'.
- Feature service computes the as-of vector (section 14.1) and inserts a row in features.
- ML service returns a probability; you insert predictions, then explanations, impacts and one recommendation for High/Medium bands.
- Dashboard reads the result for that store only.

15.3 Learning from outcomes

- When an outcome is verified (courier says delivered/RTO, or the merchant confirms), insert it into outcomes (unique per order).
- A worker finds the matching prediction, builds the labelled example, runs one validation-gated SGD step, and records model_updates. If the gate fails, the old model stays active.
- Weekly, the privileged FedAvg job aggregates category weights (sample-weighted) when enough stores participate.

15.4 Scheduled jobs

Job	Frequency	Purpose
Reconcile orders	15 min to 3 h per store	Catch missed webhooks (modified_after / updated_at)
Webhook health	Daily	Re-enable or recreate disabled WooCommerce webhooks; check Shopify subscriptions
Token refresh	Before expiry	Refresh Shopify expiring offline tokens
Churn scoring	Per store schedule (e.g. daily/weekly)	Create scoring_run, features, predictions for eligible customers
Outcome sweep	Hourly	Turn status changes / courier events into outcomes
FedAvg	Weekly	Aggregate category weights (service role only)
PII retention	Daily	Redact customers older than the retention policy; honour redact webhooks

16. Security, privacy and compliance

- **Secrets:** encrypt Shopify tokens, WooCommerce keys and webhook secrets at rest (AES-256-GCM or a managed KMS). Keep a key-id column for rotation. Never log them.
- **Transport:** HTTPS everywhere. Refuse WooCommerce stores without valid TLS.
- **Webhook verification:** verify the HMAC over the *raw* body with a constant-time comparison before parsing.
- **Input validation:** validate the Shopify shop hostname, WooCommerce store URL (block private IP ranges and localhost to prevent SSRF), and every callback parameter.
- **Least privilege:** read scopes only; minimum protected data; hash email/phone; no PII to the ML service.
- **Rate limiting and body limits** on public endpoints (webhook and callback routes).
- **Tenant isolation:** store_id from session, RLS as second defence, automated cross-tenant tests.
- **Retention and deletion:** define how long PII is kept, honour Shopify's three compliance webhooks, and provide a 'disconnect and delete' button for WooCommerce stores. India's DPDP Act 2023 applies to personal data of people in India; get a proper legal review before real merchants use the product (this guide is not legal advice).
- **Audit:** log connect, disconnect, scope changes, token refresh failures, rule/model promotions, and any cross-tenant denial.

17. A practical plan for your project

17.1 What to build first (in order)

Phase	Deliverable	How you know it works
1	Shopify dev store with 200+ test orders (CSV import or a script) including COD gateway; local or hosted WooCommerce with test orders	You can see orders in both admins
2	Shopify connector: authorization-code install, token saved encrypted, test shop { } query	Dev store connects end to end
3	WooCommerce connector: wc-auth callback, key saved, system_status call	Test store connects end to end
4	Backfill both into the normalized tables (bulk JSONL for Shopify, paged loops for Woo)	Row counts match the admin counts
5	Webhooks with HMAC check + event table + worker + reconcile job	Edit an order and see it update; replay a webhook and see no duplicate
6	Mappings screen (COD patterns, shipped status) and outcomes import (CSV)	is_cod and shipped_at are correct on test orders
7	Feature queries (14.1, 14.2) + leakage test	No feature uses data after as_of
8	Models, decision layer, dashboard (rest of your report)	Report acceptance tests pass

17.2 Getting RTO labels for a college project

- Development stores contain test data only, so RTO labels will not exist. Generate a **synthetic but realistic** dataset (documented clearly as synthetic) with COD share, state-level RTO rates and repeat-customer effects, and import it through the same connectors/CSV path so the pipeline is real even if the numbers are simulated.
- If a friendly real merchant takes part, use a labelled export from their courier panel and import it through the outcomes CSV importer. Record source = 'merchant_confirmed'.
- Do not report model scores from synthetic data as real-world performance. Your own report already states this principle.

17.3 Pre-launch checklist

- Scopes minimal; read_all_orders and protected-data approvals requested early (Shopify).
- Pinned Shopify API version in config; upgrade calendar set (quarterly).
- Compliance webhooks implemented and tested (Shopify); disconnect-and-delete flow (WooCommerce).
- Webhook handlers verify signatures, persist first, return 200 fast, and are idempotent.
- Backfills are resumable; reconcile job running; disabled-webhook repair running.
- RLS enabled with FORCE; app role has no BYPASSRLS; cross-tenant tests in CI.
- Encrypted secrets with key rotation; no secrets in logs or the browser.
- COD and shipped-status mapping confirmed per store during onboarding.

18. Sources and things to re-verify

Facts in this guide were checked on 3 October 2026 against these documents. Open them again before implementing, because versions, field names and approval rules change.

Topic	Where to look
Shopify API versions & release notes	shopify.dev/release-notes (2026-07 latest stable; 2026-10 stable from 1 Oct 2026)
Authorization code grant	shopify.dev/docs/apps/build/authentication-authorization/access-tokens/authorization-code-grant
Standalone apps & refresh tokens	shopify.dev/docs/apps/build/authentication-authorization/authenticate-standalone-apps
Client credentials grant (own organization only)	shopify.dev/docs/apps/build/authentication-authorization/client-credentials-grant
Access scopes (read_all_orders, protected customer data)	shopify.dev/api/usage/access-scopes and shopify.dev/docs/apps/launch/protected-customer-data
Bulk operations	shopify.dev/docs/api/usage/bulk-operations/queries
Mandatory compliance webhooks	shopify.dev/docs/apps/build/privacy-law-compliance
WooCommerce authentication & wc-auth	developer.woocommerce.com/docs/apis/rest-api/authentication/
WooCommerce REST API v3 resources	developer.woocommerce.com/docs/apis/rest-api/v3/
WooCommerce order attribution	woocommerce.com/document/order-attribution-tracking/

Items I could not fully confirm

- Exact GraphQL field names for fulfilment delivery times and customerJourneySummary in your pinned Shopify version: check in GraphQL.
- Which fulfilment scope Shopify requires for a specific fulfilment field: check the field's documentation page.
- Shopify's per-version concurrency limits for bulk operations and the bucket sizes per plan.
- WooCommerce's current webhook auto-disable threshold (documented here as the long-standing default of five failures).
- Whether Shopify's expiring-offline-token behaviour is mandatory for new installs in your app type; handle both token shapes.